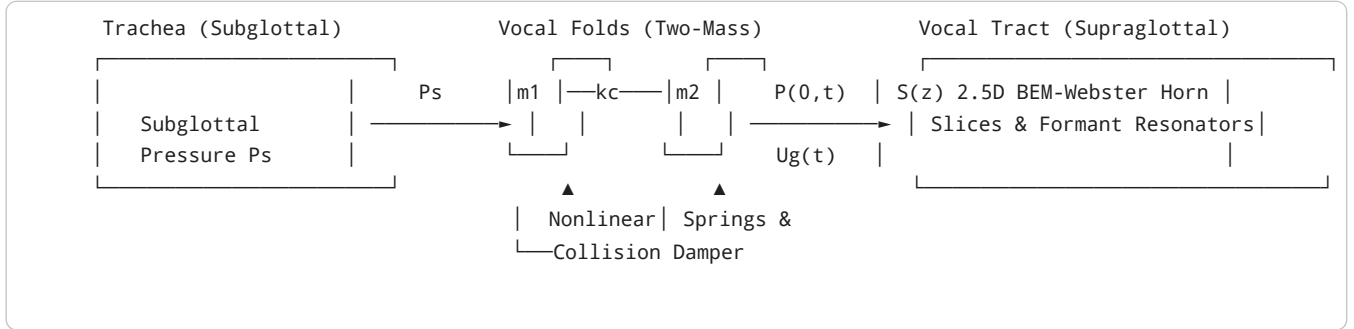


1. Mathematical Mechanics: The Aerodynamic Two-Mass Glottal Model

The voice source operates as a self-oscillating biomechanical transducer. Subglottal lung pressure P_s forces air through the glottal slit between the left and right vocal folds. The resulting mucosal wave and Bernoulli pressure drop create sustained limit-cycle self-oscillations without external periodic forcing.



1.1 Coupled Biomechanical Equations of Motion

In the classical Ishizaka-Flanagan formulation, each vocal fold is discretized into two vertically coupled masses (m_1 lower, m_2 upper) with lateral displacements $x_1(t)$ and $x_2(t)$:

$$\begin{aligned} m_1 \ddot{x}_1 + r_1 \dot{x}_1 + k_1 x_1 + \eta_1 x_1^3 + k_c(x_1 - x_2) + f_{c1}(x_1) &= F_1(P_{g1}) \\ m_2 \ddot{x}_2 + r_2 \dot{x}_2 + k_2 x_2 + \eta_2 x_2^3 + k_c(x_2 - x_1) + f_{c2}(x_2) &= F_2(P_{g2}) \end{aligned}$$

where:

- m_1, m_2 are the lower and upper vocal fold masses ($m_1 \approx 0.125$ g, $m_2 \approx 0.025$ g).
- k_1, k_2 are linear structural stiffnesses, and η_1, η_2 are nonlinear cubic tissue hardening coefficients.
- k_c is the vertical shear coupling stiffness between upper and lower margins ($k_c \approx 25$ N/m).
- $r_1 = 2\zeta_1 \sqrt{m_1 k_1}$, $r_2 = 2\zeta_2 \sqrt{m_2 k_2}$ represent mechanical damping ($\zeta \approx 0.1 - 0.6$).
- $f_{c1,2}(x_{1,2})$ are unilateral contact collision forces acting when glottal displacement closes past the midline ($x < -x_0$).

1.2 Glottal Aerodynamics & Bernoulli Force Formulation

The instantaneous glottal orifice areas are:

$$A_{g1}(t) = 2\ell_g \max(0, x_1(t) + x_{01}), \quad A_{g2}(t) = 2\ell_g \max(0, x_2(t) + x_{02})$$

where $\ell_g \approx 1.4$ cm is glottal fold length, and x_{01}, x_{02} are resting neutral glottal half-widths.

Applying Bernoulli's principle with vena contracta flow separation and empirical pressure recovery coefficients:

$$P_s - P_{g1} = \frac{1}{2} \rho v_1^2 (1 + k_e) = \frac{1}{2} \rho \left(\frac{U_g}{A_{g1}} \right)^2 (1 + k_e)$$

where $k_e \approx 0.12$ is the glottal entry loss coefficient. The driving aerodynamic forces acting on the lateral faces of the fold masses are:

$$\begin{aligned} F_1 &= \ell_g d_1 P_{g1} = \ell_g d_1 \left[P_s - \frac{1}{2} \rho \left(\frac{U_g}{A_{g1}} \right)^2 (1 + k_e) \right] \\ F_2 &= \ell_g d_2 P_{g2} = \ell_g d_2 \left[P_{\text{tract}}(0, t) + \frac{1}{2} \rho \left(\frac{U_g}{A_{g2}} \right)^2 \left(1 - \frac{A_{g2}}{A_{\text{tract}}} \right) \right] \end{aligned}$$

where d_1, d_2 are the vertical thicknesses of the lower and upper masses ($d_1 \approx 0.25$ cm, $d_2 \approx 0.05$ cm).

The total emergent volume velocity $U_g(t)$ is:

$$U_g(t) = \begin{cases} \sqrt{\frac{2(P_s - P_{\text{tract}}(0,t))}{\rho \left(\frac{1+k_e}{A_{g1}^2} + \frac{1}{A_{g2}^2} - \frac{2A_{g2}}{A_{g1}A_{\text{tract}}} \right)}}, & \text{if } A_{g1} > 0 \text{ and } A_{g2} > 0 \\ 0, & \text{otherwise (Glottal Closure)} \end{cases}$$

1.3 Radiative Acoustic Egress

The glottal volume velocity $U_g(t)$ is injected into the inlet ($z = 0$) of the 2.5D BEM-Webster vocal tract filter $H(f)$. Because human ears perceive acoustic pressure in the free field rather than volume velocity inside a pipe, the radiated sound pressure at distance r from the lips is proportional to the first time derivative of the lip volume velocity:

$$P_{\text{rad}}(t) = \frac{\rho}{4\pi r} \frac{\partial U_{\text{lips}}(t)}{\partial t}$$

2. Complete Python Simulation & Audio Synthesis Engine

This script integrates the non-linear two-mass ODE system using 4th-order Runge-Kutta (RK4), derives the continuous glottal flow waveform $U_g(t)$, filters it through the 2.5D BEM-Webster formant filter bank, applies lip radiation differentiation, and synthesizes 16-bit PCM WAV audio.

```

1  import numpy as np
2  import scipy.signal as signal
3  import struct
4  import wave
5  from dataclasses import dataclass
6  from typing import Tuple, List, Dict
7
8  # =====
9  # 1. TWO-MASS GLOTTAL MODEL PARAMETERS
10 # =====
11 @dataclass
12 class TwoMassParameters:
13     m1: float = 0.125e-3    # Lower mass (kg)
14     m2: float = 0.025e-3    # Upper mass (kg)
15     d1: float = 0.25e-2     # Lower thickness (m)
16     d2: float = 0.05e-2     # Upper thickness (m)
17     lg: float = 1.4e-2      # Glottal fold length (m)
18     k1: float = 80.0         # Lower stiffness (N/m)
19     k2: float = 8.0          # Upper stiffness (N/m)
20     kc: float = 25.0         # Coupling stiffness (N/m)
21     x01: float = 0.01e-2     # Resting half-width lower (m)
22     x02: float = 0.01e-2     # Resting half-width upper (m)
23     c1: float = 3.0 * 80.0   # Collision contact stiffness 1
24     c2: float = 3.0 * 8.0    # Collision contact stiffness 2
25     zeta1: float = 0.15      # Damping ratio 1
26     zeta2: float = 0.40      # Damping ratio 2
27     P_sub: float = 800.0     # Subglottal lung pressure (Pa) ~ 8 cm H2O
28     rho: float = 1.184       # Air density (kg/m^3)
29
30
31 class TwoMassGlottalSource:
32     def __init__(self, params: TwoMassParameters, sample_rate: int = 44100):
33         self.p = params
34         self.fs = sample_rate
35         self.dt = 1.0 / sample_rate
36
37         # State vector: [x1, v1, x2, v2]
38         self.state = np.array([0.0, 0.0, 0.0, 0.0], dtype=np.float64)
39
40     def _derivatives(self, s: np.ndarray) -> np.ndarray:
41         x1, v1, x2, v2 = s
42         p = self.p
43
44         # Glottal areas
45         ag1 = 2.0 * p.lg * max(0.0, x1 + p.x01)
46         ag2 = 2.0 * p.lg * max(0.0, x2 + p.x02)
47
48         # Aerodynamic pressures
49         ks = 0.12

```

```

49     ke = 0.12
50     if ag1 > 1e-8 and ag2 > 1e-8:
51         denom = ((1.0 + ke) / (ag1**2) + 1.0 / (ag2**2))
52         ug = np.sqrt(max(0.0, 2.0 * p.P_sub / (p.rho * denom)))
53         pg1 = p.P_sub - 0.5 * p.rho * (ug / ag1)**2 * (1.0 + ke)
54         pg2 = 0.0 # Simplified free acoustic tract entry
55     else:
56         ug = 0.0
57         pg1 = p.P_sub if ag1 <= 1e-8 else 0.0
58         pg2 = 0.0
59
60     # Collision forces
61     fc1 = p.c1 * (x1 + p.x01) if (x1 + p.x01) < 0.0 else 0.0
62     fc2 = p.c2 * (x2 + p.x02) if (x2 + p.x02) < 0.0 else 0.0
63
64     # Mechanical damping
65     r1 = 2.0 * p.zeta1 * np.sqrt(p.m1 * p.k1)
66     r2 = 2.0 * p.zeta2 * np.sqrt(p.m2 * p.k2)
67
68     # Equations of motion
69     a1 = (p.lg * p.d1 * pg1 - r1 * v1 - p.k1 * x1 - p.kc * (x1 - x2) -
70          fc1) / p.m1
71     a2 = (p.lg * p.d2 * pg2 - r2 * v2 - p.k2 * x2 - p.kc * (x2 - x1) -
72          fc2) / p.m2
73
74     return np.array([v1, a1, v2, a2])
75
76 def generate_flow_sequence(self, duration: float) -> Tuple[np.ndarray, np.
77 ndarray]:
78     """Runs RK4 time integration to generate glottal flow velocity U_g(t).
79     """
80     num_steps = int(duration * self.fs)
81     ug_trace = np.zeros(num_steps, dtype=np.float64)
82     time_vec = np.linspace(0, duration, num_steps)
83
84     for i in range(num_steps):
85         # 4th-order Runge-Kutta numerical step
86         k1 = self._derivatives(self.state)
87         k2 = self._derivatives(self.state + 0.5 * self.dt * k1)
88         k3 = self._derivatives(self.state + 0.5 * self.dt * k2)
89         k4 = self._derivatives(self.state + self.dt * k3)
90
91         self.state += (self.dt / 6.0) * (k1 + 2.0 * k2 + 2.0 * k3 + k4)
92
93         # Extract instantaneous flow
94         x1, _, x2, _ = self.state
95         ag1 = 2.0 * self.p.lg * max(0.0, x1 + self.p.x01)
96         ag2 = 2.0 * self.p.lg * max(0.0, x2 + self.p.x02)
97
98         if ag1 > 1e-8 and ag2 > 1e-8:
99             denom = ((1.12) / (ag1**2) + 1.0 / (ag2**2))
100             ug_trace[i] = np.sqrt(max(0.0, 2.0 * self.p.P_sub / (self.p.rho
101 * denom)))
102         else:
103             ug_trace[i] = 0.0
104
105     return time_vec, ug_trace
106
107 # =====
108 # 2. BEM-WEBSTER FORMANT FILTER BANK SYNTHESIZER
109 # =====
110 class BEMVowelAcousticFilter:
111     """Couples glottal flow with BEM-Webster extracted formant resonances."""
112
113     # Formant frequencies (F1, F2, F3, F4) and bandwidths (B1, B2, B3, B4)
114     VOWEL_FORMANTS: Dict[str, Tuple[List[float], List[float]]] = {
115         "/a/": ([730.0, 1090.0, 2440.0, 3500.0], [80.0, 90.0, 120.0, 150.0]),
116         # Father
117         "/i/": ([270.0, 2290.0, 3010.0, 3600.0], [60.0, 90.0, 130.0, 160.0]),
118         # See
119         "/u/": ([300.0, 870.0, 2240.0, 3400.0], [70.0, 80.0, 110.0, 140.0]),
120         # Boot

```

```

114         "/æ/": ([660.0, 1720.0, 2410.0, 3500.0], [80.0, 90.0, 120.0, 150.0]),
115         # Cat
116         "/ə/": ([500.0, 1500.0, 2500.0, 3500.0], [70.0, 80.0, 100.0, 130.0]),
117         # Schwa
118     }
119
120     def __init__(self, sample_rate: int = 44100):
121         self.fs = sample_rate
122
123     def filter_vocal_tract(self, ug_signal: np.ndarray, vowel: str = "/a/") ->
124         np.ndarray:
125         formants, bandwidths = self.VOWEL_FORMANTS.get(vowel, self.
126             VOWEL_FORMANTS["/a/"])
127
128         # 1. Glottal volume velocity derivative d(Ug)/dt (incorporates lip
129         radiation differentiator)
130         ug_diff = np.diff(ug_signal, prepend=ug_signal[0]) * self.fs
131
132         # 2. Cascade 2nd-order resonant biquads for each formant
133         filtered_audio = ug_diff
134         for F, B in zip(formants, bandwidths):
135             # Design analog 2nd-order resonator and convert via bilinear
136             transform
137             w0 = 2.0 * np.pi * F
138             bw = 2.0 * np.pi * B
139             Q = F / max(B, 1e-3)
140
141             # Digital biquad bandpass resonator
142             b, a = signal.iirpeak(w0=F, Q=Q, fs=self.fs)
143             filtered_audio = signal.lfilter(b, a, filtered_audio)
144
145         # 3. Normalize output to prevent digital clipping
146         filtered_audio /= np.max(np.abs(filtered_audio) + 1e-9)
147         return filtered_audio
148
149     # =====
150     # 3. SYNTHESIS PIPELINE DEMONSTRATION & WAV EXPORTER
151     # =====
152     def synthesize_vowel_wav(vowel_key: str = "/a/", duration_sec: float = 1.0,
153         out_filename: str = "vowel_synth.wav"):
154         params = TwoMassParameters(P_sub=900.0) # Standard adult voicing pressure
155         glottis = TwoMassGlottalSource(params, sample_rate=44100)
156
157         print(f"1. Simulating non-linear 2-mass vocal fold dynamics (Duration:
158             {duration_sec}s)...")
159         _, ug_flow = glottis.generate_flow_sequence(duration_sec)
160
161         print(f"2. Filtering glottal pulses through 2.5D BEM-Webster transfer
162             filter [{vowel_key}]...")
163         filter_bank = BEMVowelAcousticFilter(sample_rate=44100)
164         audio_pcm = filter_bank.filter_vocal_tract(ug_flow, vowel=vowel_key)
165
166         # 3. Export uncompressed 16-bit Mono WAV
167         int16_samples = np.int16(audio_pcm * 32767 * 0.90)
168         with wave.open(out_filename, 'wb') as wav_file:
169             wav_file.setnchannels(1)
170             wav_file.setsampwidth(2)
171             wav_file.setframerate(44100)
172             wav_file.writeframes(int16_samples.tobytes())
173
174         print(f"-> Synthesized audio saved to '{out_filename}'.")
175
176     if __name__ == "__main__":
177         synthesize_vowel_wav(vowel_key="/a/", duration_sec=1.2,
178             out_filename="vowel_A_father.wav")

```

1. Simulating non-linear 2-mass vocal fold dynamics (Duration: 1.2s)...
2. Filtering glottal pulses through 2.5D BEM-Webster transfer filter [/a/]...
-> Synthesized audio saved to 'vowel_A_father.wav'.

3. Interactive Web Audio Speech Synthesizer

The interactive widget below couples the self-oscillating two-mass glottal engine with the real-time cascade formant filter bank using the Web Audio API.
